

Universitatea Politehnică Timișoara
Facultatea de Automatică și Calculatoare



Proiectare cu microprocesoare

Microsistem cu microprocesorul 8086

Student: Remeșan Mihnea-Florin
Anul III, Specializarea Calculatoare
Subgrupa 3.2

Profesor coordonator: Petruc Sebastian-Ioan

An universitar: 2025-2026, Semestrul I

Tema proiectului

Să se proiecteze un microsistem cu următoarea structură:

1. Unitate centrală cu microprocesorul 8086;
2. 128 KB memorie EPROM, utilizând circuite 27C2048;
3. 64 KB memorie SRAM, utilizând circuite 62512;
4. Interfață serială, cu circuitul 8251, plasată în zona 0AF0H – 0AF2H sau 0BF0H – 0BF2H, în funcție de poziția microcomutatorului S1;
5. Interfață paralelă, cu circuitul 8255, plasată în zona 0D70H – 0D76H sau 0C70H – 0C76H, în funcție de poziția microcomutatorului S2;
6. O minitastatură cu 12 contacte;
7. 6 led-uri;
8. Un modul de afișare cu 7 segmente, cu 6 ranguri;

Toate programele în limbaj de asamblare vor fi concepute sub formă de subrutine.
Programele necesare sunt:

- rutinele de programare ale circuitelor 8251 și 8255;
- rutinele de emisie/ recepție caracter pe interfața serială;
- rutina de emisie caracter pe interfață paralelă;
- rutina de scanare a minitastaturii;
- rutina de aprindere/ stingere a unui led;
- rutina de afișare a unui caracter hexa pe un rang cu segmente.

Structura rutinelor (intrări, secvențe, ieșiri) va fi stabilită de fiecare student.

1. Unitatea Centrală cu microprocesorul 8086

Unitatea Centrală a microprocesorului 8086 este alcătuită din 3 mari componente:

- Generatorul de tact 8284A
- Microprocesorul 8086
- Circuitele pentru amplificarea și separarea magistrelor de adrese și date (circuitul amplificator/separator bidirecțional 74LS245, circuitul registru 74LS373)

Microprocesorul 8086 funcționează în mod sincron, ceea ce înseamnă că toate operațiile sale interne – aducerea instrucțiunilor din memorie (fetch), decodificarea și executarea acestora – sunt coordonate în raport cu semnalul de tact. Totuși, 8086 nu își poate genera autonom semnalele de tact și de reset, motiv pentru care necesită un circuit extern dedicat. În această arhitectură, rolul **generatorului de tact 8284A** devine esențial: acesta furnizează microprocesorului semnalele CLK, RESET și READY, asigurând sincronizarea corectă a funcționării, și generează totodată semnalul de tact pentru circuitele specializate ale interfeței seriale (capitolul 3).

Principiul de funcționare al circuitului este relativ simplu. La momentul alimentării, condensatorul C este descărcat, ceea ce determină plasarea microprocesorului în modul de reset. După un interval de ordinul zecilor de milisecunde, condensatorul se încarcă progresiv, iar semnalul RESET revine la nivelul inactiv, permițând procesorului să înceapă execuția programului. Ulterior, oscilatorul intern al 8284A generează semnalul CLK, stabilizat prin utilizarea rezonatorului cu quartz, asigurând astfel o frecvență de tact precisă și constantă.

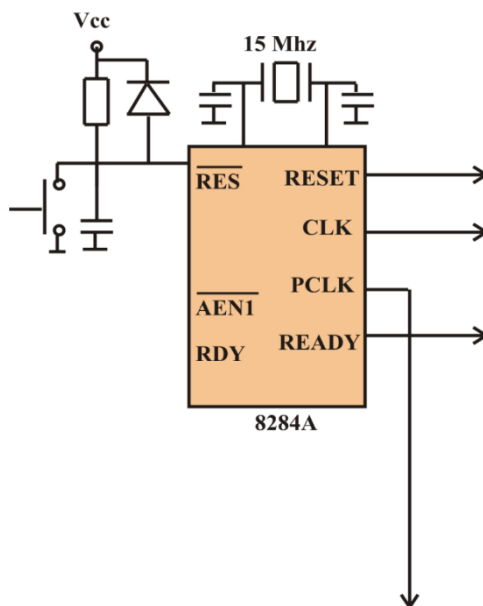


Figura 1: Generatorul de semnal 8284A

Microprocesorul 8086 reprezintă primul procesor pe 16 biți care a cunoscut o utilizare pe scară largă. Produs de Intel în anul 1978, acesta aparține **arhitecturii CISC** (Complex Instruction Set Computer) și este realizat în capsulă cu 40 de pini, fiind alimentat la o singură tensiune, $V_{cc} = 5\text{ V}$. Performanțele sale sunt susținute atât de frecvența ridicată de tact (cu

valori între 5 și 10 MHz), cât și de structura internă cu suprapunere (pipelining la nivel elementar), prin care instrucțiunile sunt preluate anticipat din memorie în perioadele în care magistralele nu sunt ocupate.

Microprocesorul poate fi utilizat într-o gamă variată de aplicații datorită celor două moduri de operare: **modul minim** și **modul maxim**. Modul minim este destinat sistemelor cu complexitate redusă, în care procesorul generează local semnalele necesare comunicării cu memoria și dispozitivele de intrare/ieșire. Modul maxim este conceput pentru sisteme complexe, inclusiv configurații multiprocesor, în care semnalele de comandă pentru memorii și porturi sunt furnizate de un controler extern de magistrală (8288). Selectarea modului de funcționare se realizează hardware, prin intermediul terminalului MN/!MX: nivelul logic „1” activează modul minim, iar nivelul logic „0” selectează modul maxim.

Structura internă cuprinde două unități interne, care lucrează asincron și independent una față de cealaltă: EU (Unitate de Execuție), care efectuează operațiile conținute codificat în instrucțiuni, și BIU (Unitatea de Interfață cu Magistrale), care are rolul de a extrage instrucțiunile din memorie și de a le decodifica.

În imaginea de mai jos, se pot observa toate semnalele microprocesorului 8086:

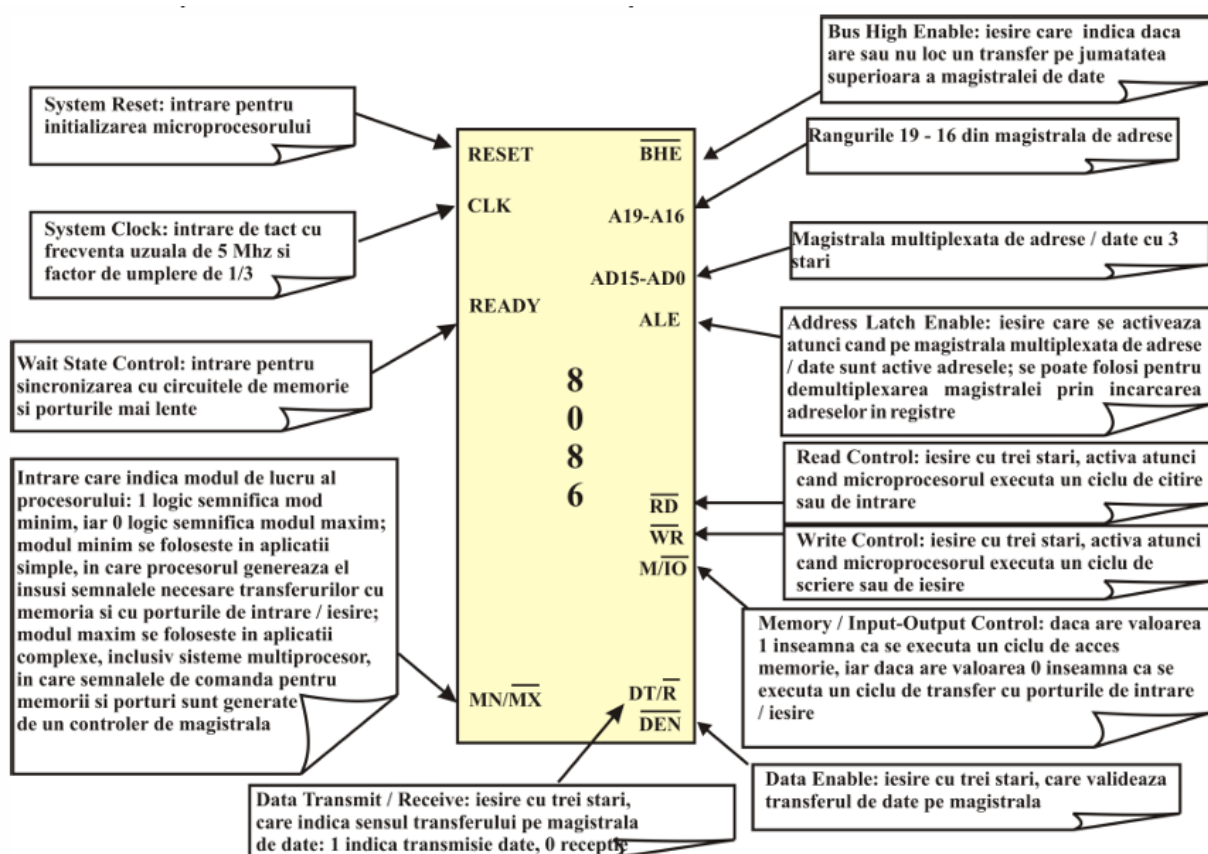


Figura 2: Principalele terminale ale microprocesorului 8086

După cum se poate observa, microprocesorul utilizează două categorii de linii: linii de adresă și linii de date. Liniile A0–A15 sunt **multiplexate**, ceea ce înseamnă că transportă atât adrese, cât și date, în funcție de faza ciclului de mașină. În schimb, liniile A16–A19 sunt **demultiplexate**, fiind dedicate exclusiv adreselor. Împreună, acestea formează o magistrală comună de 20 de biți, corespunzătoare spațiului de adrese de 1 MB. În prima fază a ciclului de magistrală, numită T1, procesorul plasează adresa pe magistrala comună, iar aceasta este stocată în latch-uri externe. În fazele următoare (T2, T3 și T4), aceeași magistrală este reutilizată pentru transferul de date între microprocesor și memorie.

În acest context apare întrebarea firească: *cum distinge memoria între faza de adresare și cea de transfer de date?* Răspunsul este oferit de semnalul **ALE** (Address Latch Enable), transmis de microprocesor către circuitele de memorie. Atunci când $ALE = 1$, liniile AD0–AD15 transportă adresa; când $ALE = 0$, aceleași linii sunt utilizate pentru date. Este important de menționat că liniile A16–A19 sunt rezervate exclusiv adresării, având rolul de a extinde adresa până la 20 de biți. Un alt semnal esențial este **BHE (Bus High Enable)**. Când $BHE = 1$, acesta activează transferul pe octetul superior al magistralei de date, iar atunci când este inactiv, selectează octetul inferior. Acest mecanism permite accesul eficient la memorii organizate pe 16 biți sau pe 8 biți.

O altă componentă esențială a unității centrale este **circuitul de tip registru** (latch) **74×373**, utilizat pentru demultiplexarea liniilor de adresă. Acesta are rolul de a transforma liniile multiplexate AD în linii stabile, dedicate exclusiv adresării, necesare accesului corect la memorie. Circuitul dispune de 8 intrări (D1–D8) și 8 ieșiri (Q1–Q8), putând astfel stoca un octet. Funcționarea sa este guvernată de două semnale de control:

- **G (Latch Enable)** – atunci când $G = 1$, ieșirile Q reproduc în timp real valorile de la intrările D; când $G = 0$, informația prezentă la un moment dat este memorată și păstrată în latch;
- **OC (Output Control)** – atunci când $OC = 0$, ieșirile Q sunt active; când $OC = 1$, ieșirile sunt trecute în stare de înaltă impedanță.

Circuitul operează, prin urmare, în trei stări posibile: nivel logic „0”, nivel logic „1” și **stare de înaltă impedanță**. Această stare reprezintă, practic, o „deconectare” a pinului de pe magistrală, astfel încât acesta nu influențează semnalul, permițând partajarea magistralei de către mai multe dispozitive fără riscul apariției conflictelor.

În cadrul sistemelor bazate pe microprocesorul 8086, latch-ul 74×373 este comandat de semnalul **ALE** (Address Latch Enable). Atunci când ALE este activ, liniile AD0–AD7 sunt capturate ca parte inferioară a adresei. Când ALE devine inactiv, valoarea stocată rămâne validă, iar magistrala AD se eliberează pentru transferul de date.

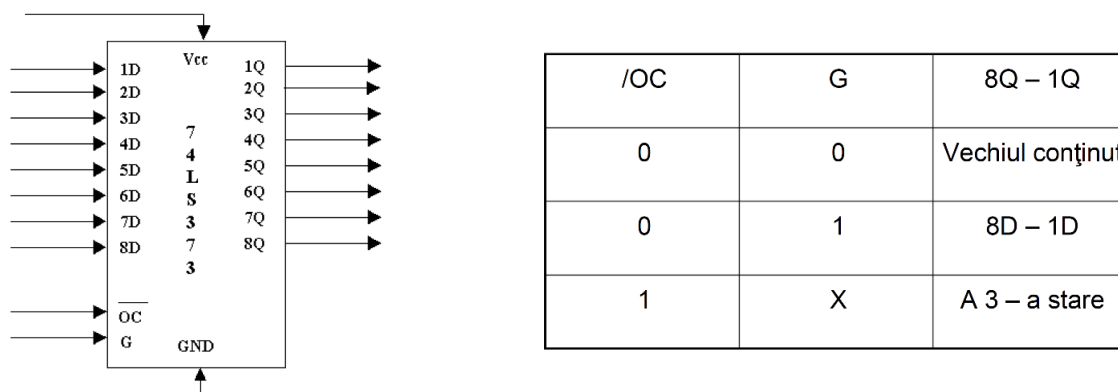


Figura 3: Latch-ul 74x373 și modul de funcționare al acestuia

O ultimă componentă a unității centrale este **circuitul amplificator/separator bidirecțional 74x245**, folosit pentru gestionarea magistralei de date. Este alcătuit din 8 perechi de porți tri-state bidirecționale, capabile să funcționeze atât ca buffere (amplificarea semnalului), cât și ca separatoare de magistrală, în funcție de necesitățile sistemului.

Circuitul dispune de două semnale principale de control:

- **/G (Output Enable)** – activează toate ieșirile atunci când este la nivel logic „0”; la nivel logic „1”, ieșirile sunt trecute în stare de înaltă impedanță, izolând magistralele.
- **DIR (Direction Control)** – stabilește direcția transferului de date:
 - DIR = 0 → date sunt preluate de pe magistrala B și transferate către A (citire)
 - DIR = 1 → date sunt transferate de pe magistrala A către B (scriere)

Prin această funcționare, 74x245 asigură amplificarea și izolarea semnalelor, prevenind conflictele pe magistrală. În sistemele cu 8086, el gestionează fluxul de date între procesor, memorie și dispozitive periferice, contribuind la stabilitatea comunicației pe magistrală.

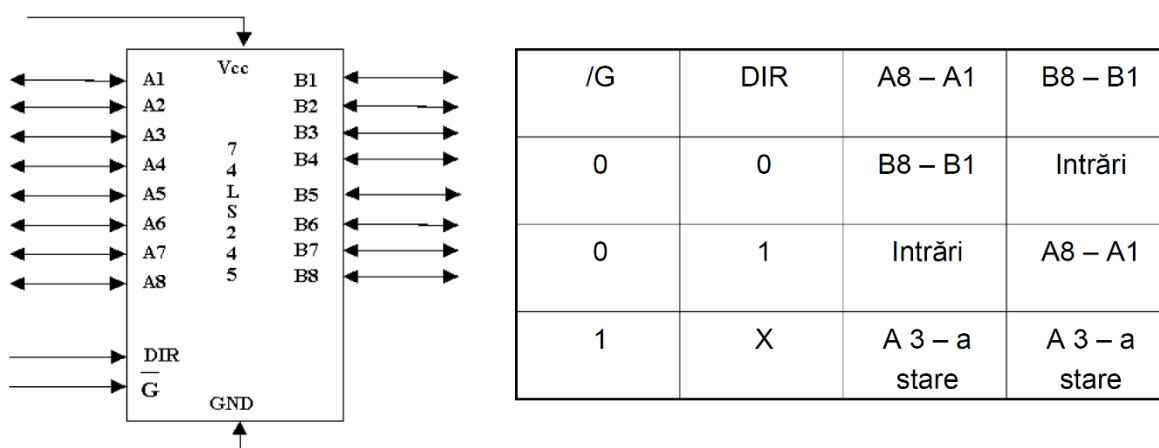


Figura 4: Circuitul amplificator 74x245 și modul de funcționare al acestuia

În imaginea de mai jos se poate vedea modul în care a fost construită Unitatea Centrală în aplicația EasyEDA. De la stânga la dreapta regăsim: generatorul de tact 8284A, microprocesorul 8086, cele 3 latch-uri 74x373 și 2 buffere bidirecționale de date 74x245.

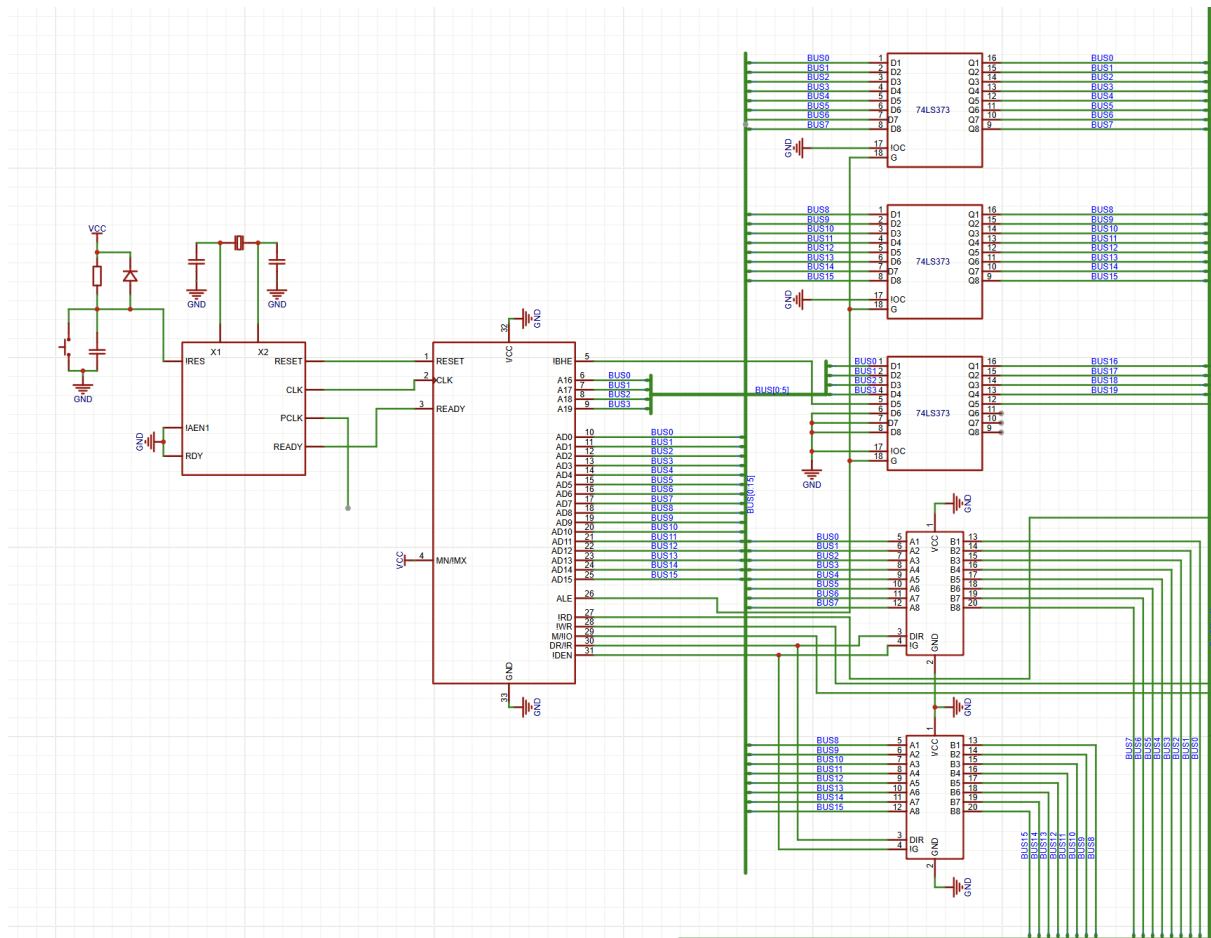


Figura 5: Unitatea Centrală în EasyEDA

2. Conectarea la memorie

Într-un sistem cu microprocesorul 8086, conectarea corectă a **memoriilor** este esențială pentru funcționarea stabilă a întregii arhitecturi. Acest proces trebuie realizat ținând cont de cele două magistrale fundamentale ale sistemului: **magistrala de adrese**, responsabilă de selectarea locațiilor din memorie, și **magistrala de date**, prin care se realizează transferul efectiv al informațiilor. Spațiul de memorie este împărțit între memoria nevolatilă, destinată stocării permanente a programului, și memoria volatilă, utilizată pentru date în timpul execuției. În cadrul acestui proiect sunt folosite două tipuri de memorii reprezentative pentru aceste funcții: **EPROM** și **SRAM**.

Memoria EPROM (Erasable Programmable Read-Only Memory) este o memorie nevolatilă, capabilă să păstreze informația chiar și în absența alimentării electrice. Structura sa internă este alcătuită din matrice de tranzistoare programabile individual, care pot fi încărcate cu date prin aplicarea unor tensiuni mai ridicate decât cele utilizate în circuitele digitale obișnuite. Odată programată, memoria EPROM poate fi ștearsă doar prin expunere la radiații ultraviolete, motiv pentru care circuitele de acest tip sunt prevăzute cu o fereastră din cuarț ce permite luminarea directă a cipului de siliciu. Expunerea la UV readuce toate celulele la starea inițială, permițând reprogramarea ulterioară a memoriei.

Memoria EPROM utilizată este reprezentată prin circuitul **27C2048**, care oferă o capacitate de 256 KB. Aceasta este conectată la arhitectura 8086 prin magistrala de adrese și magistrala de date. Pinii principali ai circuitului sunt:

- **A0 – A16** – linii de adresă (17 biți), pentru selectarea celor 128K locații de memorie.
- **Q0 – Q15** – linii de date (16 biți), prin care se transmit octeții citiți către magistrala de date a sistemului.
- **/CE (Chip Enable)** – activează circuitul; EPROM-ul răspunde la citire doar când /CE = 0.
- **/OE (Output Enable)** – validează ieșirile; la /OE = 0, datele apar pe Q0–Q7, altfel ieșirile sunt în stare de înaltă impedanță.
- **/PGM (Program Enable)** – folosit doar în faza de programare a EPROM-ului, neutilizat în funcționarea normală.
- **VPP** – pin de tensiune de programare (conectat la +5 V în exploatare normală).
- **VCC** – alimentare +5 V.
- **GND** – masă (referință).

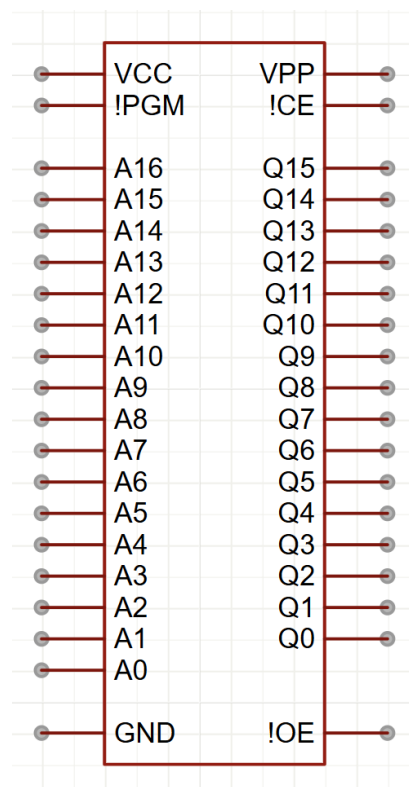


Figura 6: Circuitul EPROM 27C2048

Memoria SRAM (Static Random-Access Memory) este un tip de memorie semiconductoare volatilă, utilizată pentru stocarea rapidă a datelor în timpul execuției programului. Termenul *static* indică faptul că, spre deosebire de memoriile DRAM, conținutul nu necesită reîmprospătare periodică, fiecare bit fiind păstrat într-o structură de circuite logice stabile. Această organizare internă conferă memoriei SRAM timp de acces reduși și o funcționare fiabilă, motiv pentru care este folosită pe scară largă în sistemele cu microprocesor pentru zone de lucru rapide și pentru stocarea temporară a datelor.

Circuitul SRAM **62512** are capacitatea de memorie de 128KB, principalii pini ai circuitului fiind:

- **A0 – A15** – linii de adresă (16 biți), care permit selecția celor 65.536 locații de memorie (64 KB).
- **I/O0 – I/O7** – linii bidirecționale de date (8 biți), utilizate atât pentru citirea datelor din memorie, cât și pentru scrierea lor.
- **/CS (Chip Select)** – activează circuitul; memoria răspunde la operații doar când /CS = 0.
- **/OE (Output Enable)** – validează ieșirile în operația de citire; la /OE = 0, datele sunt plasate pe I/O0–I/O7.
- **/WE (Write Enable)** – controlează operația de scriere; când /WE = 0 și /CS = 0, datele prezente pe magistrala de date sunt stocate în locația selectată.
- **VCC** – alimentare +5 V.
- **GND** – masă (referință).

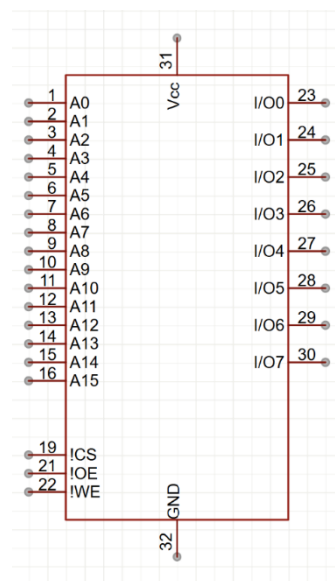


Figura 7: Circuitul SRAM 62512

Circuitul 74×138 este un decodificator de adrese de tip 3-la-8, folosit pentru selectarea zonelor de memorie în funcție de biții superiori ai magistralei de adrese. Prin activarea unei singure ieșiri dintre cele opt, permite împărțirea clară a spațiului de memorie între EPROM, SRAM și alte dispozitive, evitând conflictele pe magistrală. Principalii pini și rolurile lor sunt:

- **A2, A1, A0** – intrări de selecție (3 biți); combinația lor binară determină care dintre ieșirile Y0–Y7 este activată.
- **E3** – semnal de enable activ pe 1 logic; circuitul funcționează doar dacă E3 = 1.
- **/E2, /E1** – semnale de enable active pe 0 logic; ambele trebuie să fie 0 pentru ca decodificatorul să fie activ.
- **/Y0 – /Y7** – ieșiri decodificate **active pe 0 logic**, fiecare corespunzând unei zone distincte de memorie. Pentru fiecare combinație a intrărilor A2–A0, o singură ieșire este adusă la 0.
- **VCC** – alimentare + 5V
- **GND** – masa (referință).

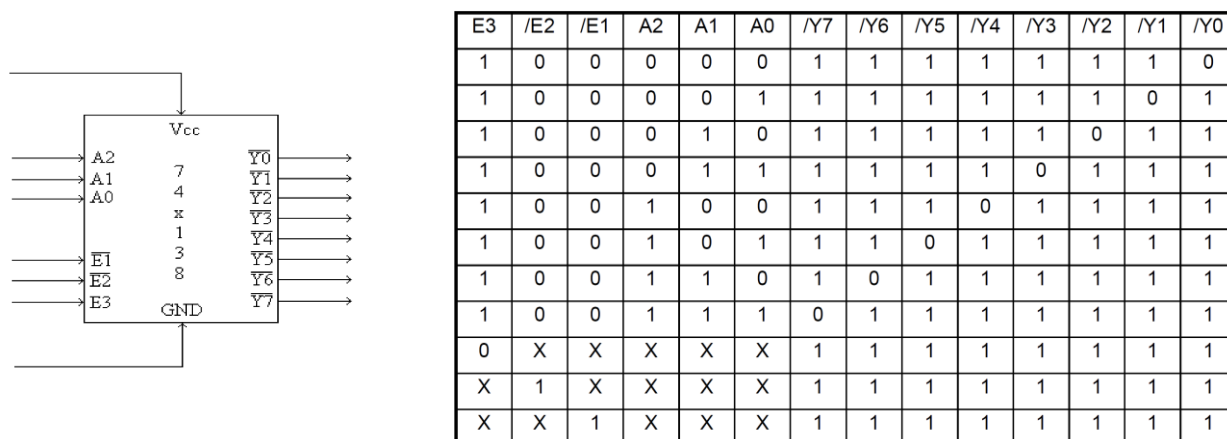


Figura 8: Circuitul decodificator 74x138 și modul de funcționare al acestuia

Pentru realizarea decodificării memoriilor, este necesar să stabilim mai întâi dimensiunea în octeți a zonelor de memorie utilizate. Având în vedere că microprocesorul 8086 dispune de **20 de biți de adresă**, spațiul de memorie este reprezentat în format hexazecimal pe **5 nibble-uri** (1 nibble = 4 biți).

- **128 KiB** = $2^7 \cdot 2^{10} = 2^{17}$ B
- **64 KiB** = $2^6 \cdot 2^{10} = 2^{16}$ B

Rezultă astfel două zone distincte în spațiul de memorie:

- zona 1: **00000H – 1FFFFH**, alocată EPROM-ului
- zona 2: **20000H – 2FFFFH**, alocată SRAM-ului

Pentru implementarea celor 128 KB de memorie EPROM, se utilizează un singur circuit 27C2048. Deși acest circuit suportă o adresare de până la 256 KB, în proiect este necesară doar jumătate din această capacitate; prin urmare, pinul A16 este conectat la GND, astfel încât să fie accesat doar primul bloc de 128 KB. EPROM-ul este conectat direct la magistrala de date și la magistrala de adrese, iar semnalul de selectare este generat prin decodificatorul 74x138.

În cazul **memoriei SRAM**, deși necesarul este de **64 KB**, un singur circuit 62512 pune la dispoziție doar 8 linii de date. Deoarece sistemul 8086 utilizează o magistrală de date pe 16 biți, este necesară implementarea memoriei în doi octeți paraleli, motiv pentru care schema folosește **două circuite 62512**, câte unul pentru octetul inferior și, respectiv, cel superior al magistralei. Cele două blocuri sunt **adresate în paralel**, dar fiecare deservește doar 8 biți ai magistralei de date. Pentru a evita maparea simultană a celor 16 biți, pinul /CS al fiecărui circuit 62512 permite activarea separată a fiecărui bloc. Astfel, cele două memorii pot fi selectate independent pentru partea inferioară sau superioară a cuvântului, în funcție de semnalele **A0** și **BHE** generate de microprocesorul 8086.

Decodificarea completă a memoriei se poate vizualiza în tabelul de mai jos:

Zona	A19	A18	A17	A16	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Tabelul 1: Decodificarea adreselor de memorie

În urma decodificării biților superiori ai magistralei de adrese pentru cele două zone de memorie, sunt obținute următoarele semnale de selecție:

$$SEL_1 = \overline{A_{19}} + \overline{A_{18}} + \overline{A_{17}} = \overline{Y_0} \text{ (marcat cu bej în tabel)}$$

$$SEL_2 = \overline{A_{19}} + \overline{A_{18}} + A_{17} + \overline{A_{16}} = \overline{Y_1} \text{ (marcat cu verde în tabel)}$$

În imaginea de mai jos este prezentată conectarea memoriei realizată în EasyEDA. Magistrala din partea stângă, care se continuă în partea superioară a schemei, reprezintă magistrala de adrese, iar liniile dispuse în partea inferioară și în dreapta fiecărui bloc de memorie constituie magistrala de date.

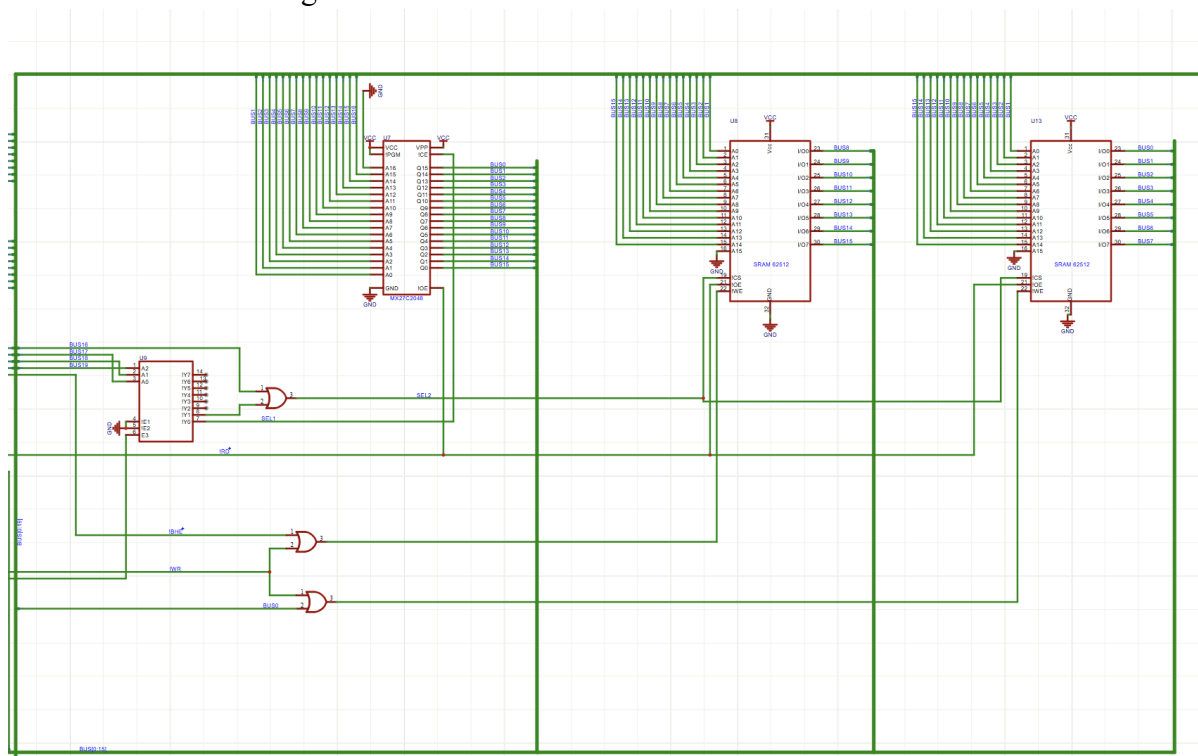


Figura 9: Conectarea la memorie în EasyEDA

3. Interfața serială și paralelă

O **interfață** este un ansamblu de mecanisme care permit comunicarea dintre două componente ale unui sistem, definind modul, formatul și regulile prin care informația este transmisă. În arhitectura 8086, termenul se referă la dispozitive hardware specializate, precum 8251 și 8255, care controlează **schimbul de date între procesor și periferice**, adaptează vitezele de lucru, asigură semnalele de sincronizare și gestionează modul de transfer, fie el serial sau paralel.

Interfața serială este ansamblul de circuite care permite transmiterea datelor bit cu bit între unitatea centrală și dispozitivele periferice. Acest tip de transfer este preferat pentru comunicațiile pe distanțe mai mari, deoarece necesită un număr redus de fire și oferă o rezistență crescută la interferențe, ceea ce reduce costurile de conectare și crește fiabilitatea.

Circuitul specializat pentru această funcție, **8251**, este un USART (Universal Synchronous/Asynchronous Receiver/Transmitter) capabil să convertească datele paralele primite de la CPU în flux serial, respectiv să transforme datele seriale primite din exterior în format paralel pentru procesor. 8251 poate funcționa atât în mod sincron, cât și în mod asincron, adaptându-se diferitelor protocoale de comunicație serială.

Funcționarea interfeței seriale 8251 este determinată de combinația liniilor de control /CS, /RD, /WR și C/D. În funcție de acestea, magistrala de date poate fi tri-stată, iar CPU poate citi sau scrie fie octeți de stare, fie date, fie cuvinte de comandă.

Cei mai importanți **pini** sunt prezentați mai jos:

- /CS – Activează circuitul; când este 0, 8251 răspunde la comenzi și date, iar când este 1, ieșirile sunt în stare High-Z.
- /RD – Semnal de citire activ pe 0; permite procesorului să citească registrul de stare sau registrul de date.
- /WR – Semnal de scriere activ pe 0; permite procesorului să scrie comenzi sau date în 8251.
- C/D – Selectează registrul accesat: 1 pentru comandă, 0 pentru date.
- D0–D7 – Magistrala de date pe 8 biți prin care CPU transferă sau primește octeți.
- TxD – Linia pe care 8251 transmite serial datele către periferic.
- RxD – Linia prin care 8251 primește serial date din exterior.
- TxDY – Indică faptul că 8251 este pregătit să primească un nou octet pentru transmisie.
- RxDY – Indică faptul că 8251 a recepționat un octet complet și poate fi citit de CPU.
- TxC – Ceasul care stabilește viteza de transmitere serială (baud rate).
- RxC – Ceasul care stabilește viteza de recepție serială.
- /RESET – Resetează circuitul și îl readuce în starea inițială.
- /CTS – Semnal din partea perifericului care spune dacă este gata să primească date.
- /RTS – Semnal emis către periferic pentru a solicita permisiunea de transmitere.

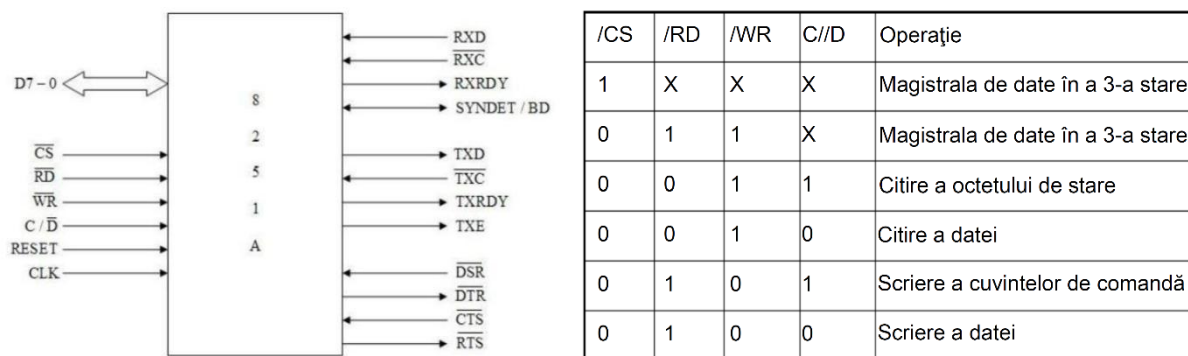


Figura 10: Circuitul 6251 și modul de funcționare al acestuia

Spre deosebire de transferul serial, unde datele sunt transmise bit cu bit, în **transferul paralel** cei 8 biți ai unui octet sunt transferați simultan, împreună cu semnalele de control necesare dialogului dintre unitatea centrală și periferice. Această abordare permite o viteză mai mare de transfer, fiind utilizată în special pentru comunicațiile pe distanțe scurte, unde numărul mai ridicat de linii fizice nu reprezintă o limitare majoră.

Comunicarea cu magistrala de date a unității centrale se realizează prin intermediul bufferului de date al circuitului 8255, utilizând liniile D0–D7, prin care sunt transmise atât datele propriu-zise, cât și cuvintele de control sau stare.

Pentru a permite transferul corect al informațiilor, circuitul 8255 utilizează un set de pini esențiali, organizați pe categorii:

Pini pentru comunicarea cu procesorul:

- **D7–D0** – Magistrala de date pe 8 biți prin care CPU citește sau scrie date, comenzi și stare.
- **/CS – Chip Select**; activează circuitul. Când este 0, 8255 răspunde la /RD și /WR.
- **/RD – Read**; activ pe 0. CPU citește datele din porturi sau registrul de stare.
- **/WR – Write**; activ pe 0. CPU scrie date sau cuvinte de control în 8255.
- **A1, A0** – Selectează regiștrii: determină ce port sau registru este accesat:
 - A1 A0 = 00 → Port A
 - A1 A0 = 01 → Port B
 - A1 A0 = 10 → Port C
 - A1 A0 = 11 → RCC
- **RESET** – Resetează circuitul, configurând toate porturile ca intrări.

Pini pentru comunicarea cu perifericele:

- **PA7–PA0** – Port A (8 biți), utilizat pentru transferul paralel de date sau semnale.
- **PB7–PB0** – Port B (8 biți), configurabil ca intrare/ieșire.
- **PC7–PC0** – Port C (8 biți), folosit fie ca port de date, fie pentru semnalele de control în modurile 1 și 2.

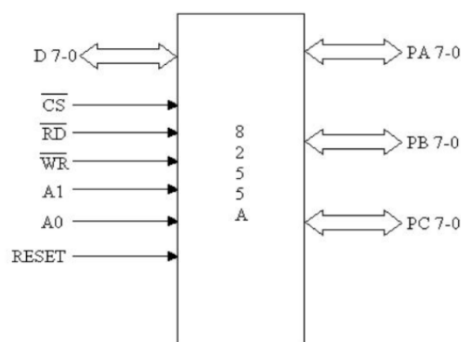


Figura 11: Circuitul 8255

/CS	/RD	/WR	A1	A0	Operația
0	1	0	0	0	Scrisoare în portul A
0	1	0	0	1	Scrisoare în portul B
0	1	0	1	0	Scrisoare în portul C
0	1	0	1	1	Scrisoare în portul cuvântului de comandă
0	0	1	0	0	Citire din portul A
0	0	1	0	1	Citire din portul B
0	0	1	1	0	Citire din portul C
0	0	1	1	1	Fără operație – magistrala de date este în a 3-a stare
0	1	1	x	x	Fără operație – magistrala de date este în a 3-a stare
1	x	x	x	x	Magistrala de date este în a 3-a stare

Figura 12: Modul de funcționare a circuitului 8255

Circuitul decodificator 74×138 este utilizat într-un mod similar cu cel folosit pentru memorie, diferența constând în modul de configurare al semnalelor de activare. Semnalul **E3** este conectat la un nivel logic „1” permanent, **/E2** este comandat de semnalul M/!IO provenit de la microprocesorul 8086, iar **/E1** este utilizat pentru aplicarea biților „0” necesari procesului de decodificare. În ceea ce privește intrările de selecție **A2**, **A1** și **A0**, acestea sunt alimentate cu liniile de adresă A10, A9 și A8, conform tabelului de decodificare.

Circuitul MAX232 este un transceiver RS-232 ↔ TTL utilizat pentru adaptarea nivelurilor de tensiune între circuite digitale și interfețe seriale RS-232.

Caracteristici:

- conversie niveluri TTL (0–5 V) ↔ RS-232 (±10...±12 V);
- două drivere și două receptoare integrate;
- alimentare la +5 V, fără tensiuni externe;
- utilizarea condensatorilor externi pentru generarea tensiunilor;
- interfațare frecventă cu circuite seriale precum **8251A**.

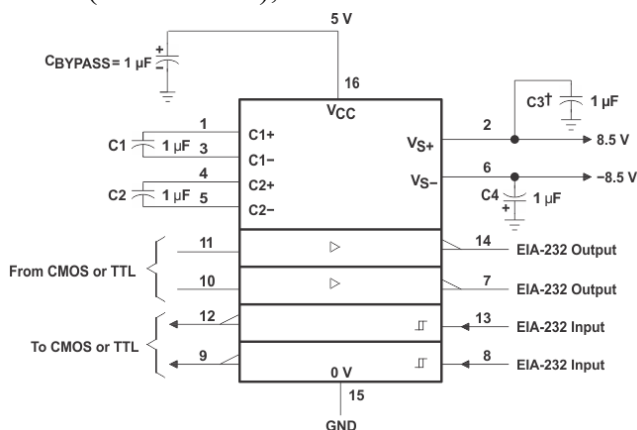


Figura 13: Circuitul MAX232

Circuitul 8253 este un contor–temporizator programabil, folosit pentru generarea de semnale de tact, temporizări și numărarea de evenimente.

Utilizări principale:

- generarea semnalelor de tact;
- obținerea de întârzieri programabile;
- numărarea evenimentelor externe;
- realizarea de ceasuri și semnale periodice.

În acest proiect, 8253 este utilizat pentru a genera semnalele de tact RxC și TxC necesare funcționării asincrone a circuitului 8251, asigurând viteza de transmisie dorită.

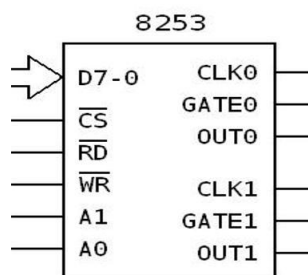


Figura 14: Circuitul 8253

Conectarea porturilor se realizează pe baza decodificării anumitor biți de adresă. Este important de menționat că această decodificare este incompletă (vezi tabelul de pe pagina următoare), deoarece biții A2, A1 și A0 nu sunt utilizați în mod intenționat, întrucât structura aplicației nu necesită implicarea acestora în procesul de selectare a porturilor.

Ecuatii enable-uri decodificator:

$$E_3 = A_{11} \cdot A_6 \cdot A_5 \cdot A_4$$

$$\overline{E_2} = M / \overline{I/O}$$

$$\overline{E_1} = \overline{A_{15}} + \overline{A_{14}} + \overline{A_{13}} + \overline{A_{12}} + \overline{A_3}$$

Se aleg arbitrar biții A10, A9 și A8 pentru ieșirile decodificatorului 138. Conform funcționării acestuia, vom avea active pe „0” logic patru ieșiri: $\overline{Y_2}, \overline{Y_3}$ pentru interfața serială, respectiv $\overline{Y_4}, \overline{Y_5}$ pentru interfața paralelă.

Ecuatii ieșiri de selecție:

$$SEL_SERIAL = \overline{Y_2} + \overline{Y_3}$$

$$SEL_PARALEL = \overline{Y_4} + \overline{Y_5}$$

Bitul A7 reprezintă trecerea de la interfața serială la paralelă:

- A7 = 1 – serială
- A7 = 0 – paralelă

		A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	
S1	0AF0H	0	0	0	0	1	0	1	0	1	1	1	1	0	0	0	0	Port date
	0AF2H	0	0	0	0	1	0	1	0	1	1	1	1	0	0	1	0	Port stare/comenzi
	0BF0H	0	0	0	0	1	0	1	1	1	1	1	1	0	0	0	0	Port date
	0BF2H	0	0	0	0	1	0	1	1	1	1	1	1	0	0	1	0	Port stare/comenzi
S2	0D70H	0	0	0	0	1	1	0	1	0	1	1	1	0	0	0	0	Port A
	0D72H	0	0	0	0	1	1	0	1	0	1	1	1	0	0	1	0	Port B
	0D74H	0	0	0	0	1	1	0	1	0	1	1	1	0	1	0	0	Port C
	0D76H	0	0	0	0	1	1	0	1	0	1	1	1	0	1	1	0	RCC
	0C70H	0	0	0	0	1	1	0	0	0	1	1	1	0	0	0	0	Port A
	0C72H	0	0	0	0	1	1	0	0	0	1	1	1	0	0	1	0	Port B
	0C74H	0	0	0	0	1	1	0	0	0	1	1	1	0	1	0	0	Port C
	0C76H	0	0	0	0	1	1	0	0	0	1	1	1	0	1	1	0	RCC

Tabelul 2: Decodificarea adreselor interfețelor serială și paralelă și stabilirea porturilor

Rutinele pentru interfața serială și paralelă sunt necesare pentru a asigura comunicarea corectă și sincronizată între microprocesor și periferice, respectând regulile impuse de circuitele de interfață. Prin utilizarea rutinelor, se verifică starea dispozitivelor înainte de transmiterea sau recepția datelor, evitând pierderea informației. În plus, rutinele permit organizarea clară și reutilizarea codului, separând logica aplicației de detaliile hardware.

Rutina de programare 8251:

init8251:

```

MOV DX, 0AF2H/0BF2H    ;port de stare/comenzi
MOV AL, 0CEH           ;cuvânt de mod
OUT DX, AL
MOV AL, 15H            ;15H = cuvânt de comandă pentru 8251
OUT DX, AL

```

RET

Rutina de programare 8255:

init8255:

```

MOV DX, 0D76H/0C76H
MOV AL, 98H            ;cuvânt de comandă pentru 8255
OUT DX, AL

```

RET

Rutina de emisie caracter pe interfața serială:

serialTransmit8251:

MOV DX, 0AF2H/0BF2H

Wait:

IN AL, DX

;citește cuvântul de stare

TEST AL, 01H

;verifică bitul 0 = TxRDY să vadă dacă s-a

transmis ultimul octet

JZ Wait

MOV DX, 0AF0H/0AF2H

;selectează registrul de date

MOV AL, CL

;încarc caracterul de transmis (ca parametru)

OUT DX, AL

RET

Rutina de recepție caracter pe interfața serială:

serialReceive8251:

MOV DX, 0AF2H/0BF2H

;selectează registrul de stare

Wait:

IN AL, DX

TEST AL, 02H

;verifică bitul RxRDY să vadă dacă există un octet

complet

JZ Wait

;așteaptă până când RxRDY = 1

MOV DX, 0AF0H/0AF2H

;selectează registrul de date

IN AL, DX

;citește caracterul recepționat

MOV CL, AL

;salvează caracterul

RET

Rutina de emisie caracter pe interfața paralelă:

parralelTransmit8255:

MOV DX, 0D74H/0C74H

;PCH = port de intrare

Wait:

IN AL, DX

;citește starea perifericului

TEST AL, 10H

;verifica BUS Busy, 10 = mască pentru PCH

JNZ Wait

MOV DX, 0D72H/0C72H

;selectează port B = de date

MOV AL, CL

;încarcă caracterul

OUT DX, AL

;trimite datele

RET

18

4. Conectarea afișajelor și a minitastaturii

Minitastatura utilizată în cadrul microsistemului are o **structură matriceală 4×3**, rezultând un total de **12 contacte**. La intersecția liniilor cu coloanele se află tastele, această organizare permițând reducerea numărului de linii de interfață necesare conectării la unitatea centrală.

Tastatura este conectată prin intermediul a două porturi: **un port de ieșire**, utilizat pentru activarea coloanelor, și **un port de intrare**, utilizat pentru citirea stării liniilor. Portul de ieșire este prevăzut cu memorare, iar portul de intrare este realizat cu porți în trei stări.

Funcționarea se bazează pe **scanarea matriceală**. Pe rând, se aplică nivel logic „0” pe câte o coloană, în timp ce celelalte coloane sunt menținute la nivel logic „1”, după care se citesc liniile. Detectarea nivelului logic „0” pe o linie indică acționarea tastei aflate la intersecția dintre linia respectivă și coloana activată.

Porturile utilizate sunt:

- **ST1 = 0000H** – port de intrare (linii);
- **ST2 = 0100H** – port de ieșire (coloane).

Selecția acestora se realizează prin decodificarea liniilor de adresă, fiind utilizată adresarea incompletă:

$$SEL_ST1 = \overline{A_{11}} \cdot \overline{A_{10}} \cdot \overline{A_9} \cdot \overline{A_8}$$

$$SEL_ST2 = \overline{A_{11}} \cdot \overline{A_{10}} \cdot \overline{A_9} \cdot A_8$$

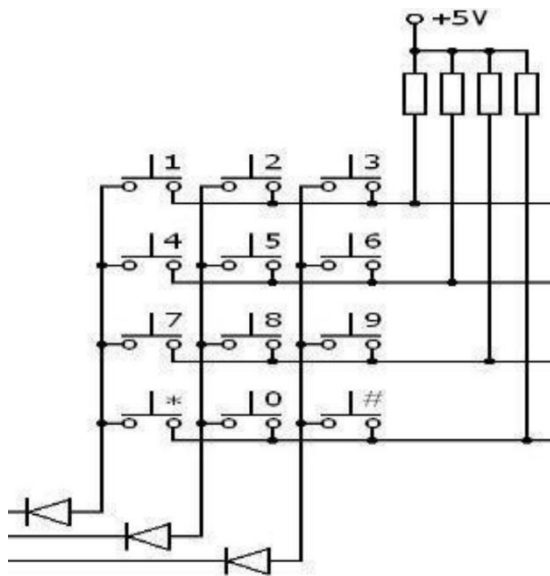


Figura 16: Minitastatura cu 12 contacte

Circuitul 74LS244 este un buffer octal cu ieșiri tristate, utilizat pentru a amplifica semnale și pentru a izola diferite secțiuni ale unui sistem digital. Conține 8 buffer-e neîncrucișate, grupate în două blocuri a câte 4 biți, fiecare controlat de un semnal propriu de Output Enable.

Caracteristici principale:

- Funcționează ca driver de magistrală, permițând conectarea mai multor dispozitive pe aceeași linie fără conflicte.
- Ieșiri tri-state (HIGH, LOW, Hi-Z), utile pentru partajarea magistralelor de date.
- Amplifică și protejează liniile de date împotriva suprasarcinii.
- Disponibil în tehnologie TTL din familia LS, cu viteză crescută și consum redus.

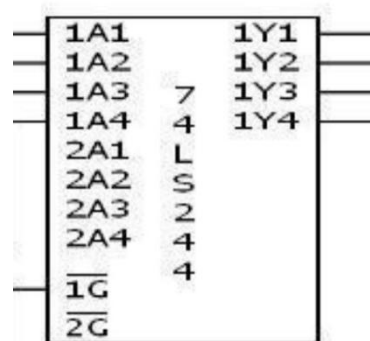


Figura 17: Circuitul 74LS244

LED-ul (Light Emitting Diode) este un dispozitiv semiconductor care emite lumină atunci când joncțiunea p-n este polarizată direct. În cadrul microsistemului, LED-urile sunt conectate astfel încât aprinderea acestora se realizează prin aplicarea nivelului logic „0” la ieșirea portului, LED-urile fiind astfel **active pe nivel logic scăzut**.

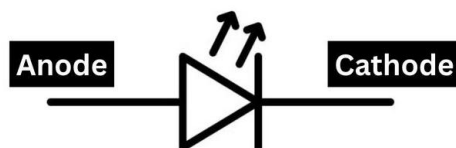


Figura 18: Simbolul unui LED

Pentru menținerea stării LED-urilor (aprins sau stins) se utilizează registre de tip **74LS373**, conectate la magistrala de date. Aceste registre permit memorarea valorii scrise de microprocesor, asigurând o comandă stabilă a LED-urilor.

Cele **6 LED-uri** sunt împărțite într-un singur grup, portul folosit fiind **0600H**, având următoarea ecuație: $SEL_SL1 = \overline{A_{11}} \cdot A_{10} \cdot A_9 \cdot \overline{A_8}$

Adresarea utilizată este **incompletă**, deoarece nu sunt decodate toate liniile de adresă, această soluție fiind adoptată pentru simplificarea structurii hardware.

Afișajul cu 7 segmente utilizat în cadrul microsistemului este organizat pe **6 ranguri**, fiecare rang fiind format din cele șapte segmente și punctul zecimal. Segmentele corespunzătoare tuturor rangurilor sunt conectate în comun, iar afișarea caracterelor se realizează prin **multiplexare în timp**. Aceasta este o tehnică prin care mai multe elemente sunt comandate succesiv, pe rând, folosind aceleași linii hardware, fiecare fiind activ doar pentru un interval foarte scurt de timp.

Comanda segmentelor se face prin intermediul unui port de ieșire conectat la magistrala de date, valorile scrise fiind memorate cu ajutorul unui registru de tip **74LS373**. Fiecare combinație de biți aplicată pe acest port corespunde unui anumit caracter afișat (cifră sau simbol hexazecimal).

Selectarea rangurilor se realizează separat, prin linii dedicate, fiecare rang fiind activat pe rând. La un moment dat este activ un singur rang, iar datorită vitezei mari de comutare, caracterele apar afișate simultan pentru observator.

Pentru funcționarea corectă a afișajului se utilizează rezistențe de limitare a curentului pe fiecare segment. Aprinderea segmentelor se face prin aplicarea nivelului logic corespunzător tipului de afișaj utilizat.

Adresarea porturilor asociate afișajului se realizează prin decodarea liniilor de adresă, utilizând semnale dedicate de selecție, fiind adoptată **adresarea incompletă**, soluție care permite simplificarea structurii hardware.

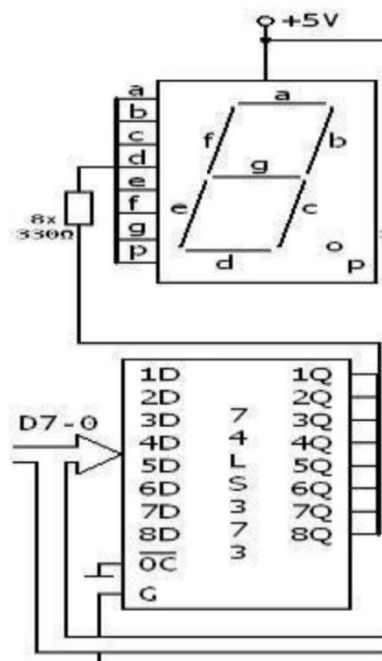


Figura 19: Afișajul cu 7 segmente

Porturile utilizate sunt:

- SA1 = 0800H
- SA2 = 0900H
- SA3 = 0A00H
- SA4 = 0B00H
- SA5 = 0C00H
- SA6 = 0D00H

Selecția acestora se realizează prin decodificarea liniilor de adresă, fiind utilizată adresarea incompletă:

$$SEL_SA1 = A_{11} \cdot \overline{A_{10}} \cdot \overline{A_9} \cdot \overline{A_8}$$

$$SEL_SA2 = A_{11} \cdot \overline{A_{10}} \cdot \overline{A_9} \cdot A_8$$

$$SEL_SA3 = A_{11} \cdot \overline{A_{10}} \cdot A_9 \cdot \overline{A_8}$$

$$SEL_SA4 = A_{11} \cdot \overline{A_{10}} \cdot A_9 \cdot A_8$$

$$SEL_SA5 = A_{11} \cdot A_{10} \cdot \overline{A_9} \cdot \overline{A_8}$$

$$SEL_SA6 = A_{11} \cdot A_{10} \cdot \overline{A_9} \cdot A_8$$

În tabelul de mai jos, se pot observa mai bine toate adresele decodificate:

	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	
0000H	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	ST1
0100H	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	ST2
0600H	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	SL1
0800H	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	SA1
0900H	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	SA2
0A00H	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	SA3
0B00H	0	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	SA4
0C00H	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	SA5
0D00H	0	0	0	0	1	1	0	1	0	0	0	0	0	0	0	0	SA6

Tabelul 3: Decodificarea adreselor minitastaturii, LED-urilor și afișajelor

Rutina de scanare minitastatură:

scanKeypad:

;scanare prima coloană

MOV AL, 0FEH

;activează prima coloană

OUT 0100H, AL

;0100H = portul de ieșire al coloanelor

IN AL, 0000H

;0000H = portul de intrare al liniilor

TEST AL, 01H

;testez prima linie – 0001

JZ Tasta1

TEST AL, 02H

;0010

JZ Tasta4

TEST AL, 04H

;0100

JZ Tasta7

TEST AL, 08H

JZ Tasta*

;scanare a doua coloană

MOV AL, 0FDH

;activează a doua coloană

OUT 0100H, AL

IN AL, 0000H

TEST AL, 01H

JZ Tasta2

TEST AL, 02H

JZ Tasta5

TEST AL, 04H

JZ Tasta8

```

TEST AL, 08H
JZ Tasta0

;scanare a treia coloană
MOV AL, 0FBH
OUT 0100H, AL
IN AL, 0000H
TEST AL, 01H
JZ Tasta3
TEST AL, 02H
JZ Tasta6
TEST AL, 04H
JZ Tasta9
TEST AL, 08H
JZ Tasta#
MOV AL, 00H                ;nici o tasta apăsată
RET

Tasta1: MOV AL, 1
        RET
Tasta2: MOV AL, 2
        RET
Tasta3: MOV AL, 3
        RET
Tasta4: MOV AL, 4
        RET
Tasta5: MOV AL, 5
        RET
Tasta6: MOV AL, 6
        RET
Tasta7: MOV AL, 7
        RET
Tasta8: MOV AL, 8
        RET
Tasta9: MOV AL, 9
        RET
Tasta*: MOV AL, *
        RET
Tasta0: MOV AL, 0
        RET
Tasta#: MOV AL, #
        RET

```

Rutina de aprindere a LED-urilor:

Pentru ca avem anod comun, LED-ul va fi aprins cand „0” logic este activ!

turnOnLED:

MOV DX, 0600H

MOV AL, XX

OUT DX, AL

RET

Pentru aprinderea unui LED dorit, vom urmări tabelul de mai jos:

	L8	L7	L6	L5	L4	L3	L2	L1	XX
LED1	1	1	1	1	1	1	1	0	0FEH
LED2	1	1	1	1	1	1	0	1	0FDH
LED3	1	1	1	1	1	0	1	1	0FBH
LED4	1	1	1	1	0	1	1	1	0F7H
LED5	1	1	1	0	1	1	1	1	0EFH
LED6	1	1	0	1	1	1	1	1	0DFH
Toate	1	1	0	0	0	0	0	0	0C0H

Rutina de stingere a LED-urilor:

turnOffLED:

MOV DX, 0600H

MOV AL, FFH

;FFH = stingere LED

OUT DX, AL

RET

Rutina de afișare a unui caracter hexazecimal:

Având un modul de afișare cu 7 segmente, putem ușor să ne folosim de acestea pentru a reprezenta caracterele 0-F în format hexazecimal. Tabelul codificării va conține, pe lângă cele 7 segmente (notate a, b, c, d, e, f, g), și un al 8-lea bit – DP = Decimal Point. De menționat este și faptul că pe „0” logic LED-ul este aprins, iar pe „1” logic este stins.

	DP	g	f	e	d	c	b	a	
0	1	1	0	0	0	0	0	0	0C0H
1	1	1	1	1	1	0	0	1	0F9H
2	1	0	1	0	0	1	0	0	0A4H
3	1	0	1	1	0	0	0	0	0B0H
4	1	0	0	1	1	0	0	1	099H
5	1	0	0	1	0	0	1	0	092H
6	1	0	0	0	0	0	1	0	082H
7	1	1	1	1	1	0	0	0	0F8H
8	1	0	0	0	0	0	0	0	080H
9	1	0	0	1	0	0	0	0	090H
A	1	0	0	0	1	0	0	0	088H
B	1	0	0	0	0	0	1	1	083H
C	1	1	0	0	0	1	1	0	0C6H

D	1	0	1	0	0	0	0	1	0A1H
E	1	0	0	0	0	1	1	0	086H
F	1	0	0	0	1	1	1	0	08EH

.data

```
CODES DB 0C0H, 0F9H, 0A4H, 0B0H, 099H, 092H, 082H, 0B8H, 080H, 090H
        DB 088H, 083H, 0C6H, 0A1H, 086H, 08EH
DISPLAY DB 0, 1, 2, 10, 14, 15
PORTS DW 0800H, 0900H, 0A00H, 0B00H, 0C00H, 0D00H
```

.text

start:

```
MOV BX, OFFSET CODES      ;BX – adresa de început a tabelului CODES
```

mainLoop:

```
CALL displayLoop
```

```
CALL delay                ;delay de multiplexare
```

```
JMP mainLoop
```

displayLoop:

```
MOV SI, OFFSET DISPLAY    ;SI – adresa de început a tabelului DISPLAY
```

```
MOV DI, OFFSET PORTS      ;DI – adresa de început a tabelului CODES
```

```
MOV CX, 6                  ;număr de iterații – control pentru Loop
```

nextDigit:

```
MOV AL, [SI]              ;AL – indexul din DISPLAY
```

```
XLAT                      ;transformă indexul în cod 7-segment (ex: 10 – „A”)
```

```
MOV DX, [DI]              ;DX – adresa rangului portului
```

```
OUT DX, AL                ;afișare caracter pe rang
```

```
INC SI                    ;următorul byte (DB) din DISPLAY
```

```
ADD DI, 2                  ;următorul word (DW) din PORTS
```

```
LOOP nextDigit
```

```
RET
```

delay:

```
MOV CX, 3000
```

d1:

```
LOOP d1
```

```
RET
```

Pentru afișarea corectă a caracterelor pe afișajul cu 7 segmente multiplexat este necesară introducerea unui delay de temporizare. Deoarece la un moment dat este activ un singur rang, fiecare rang trebuie menținut activ pentru un interval scurt de timp, astfel încât ochiul uman să perceapă afișarea ca fiind continuă. Fără acest delay, comutarea rangurilor s-ar face prea rapid, ceea ce ar conduce la o afișare instabilă sau imperceptibilă. Delay-ul de multiplexare asigură persistența vizuală și stabilizează afișajul.

În imaginea de mai jos, se poate observa conectarea afișajelor și a minitastaturii:

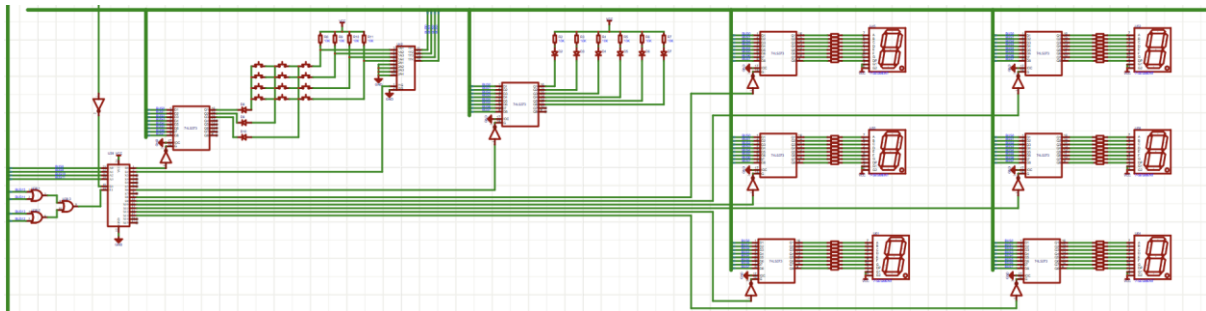


Figura 20: Conectarea afișajelor și a minitastaturii în EasyEDA

Bibliografie

Materialele din curs și laborator

Intel Corporation, *Intel 8086 Microprocessor – Datasheet – official Intel technical documentation*, (accesat la data de 25.10.2025).

GeeksforGeeks, *Architecture of 8086 Microprocessor – online educational resource*, (accesat la data de 25.10.2025).

Intel Corporation, *Memory Interfacing with the 8086/8088 – official Intel technical documentation*, (accesat la data de 13.11.2025).

GeeksforGeeks, *Memory Interfacing in 8086 – online educational resource*, (accesat la data de 13.11.2025).

Intel Corporation, *Intel 8251A USART – Datasheet – official Intel technical documentation*, (accesat la data de 27.11.2025).

GeeksforGeeks, *8255 Programmable Peripheral Interface (PPI) – online educational resource*, (accesat la data de 27.11.2025).

Texas Instruments, *SN7447 BCD to Seven-Segment Decoder/Driver – Datasheet – official Texas Instruments technical documentation*, (accesat la data de 11.12.2025).

GeeksforGeeks, *Matrix Keypad – Scanning and Interfacing – online educational resource*, (accesat la data de 11.12.2025).